Kubernetes security deep dive

Threat modelling, identities and tokens, RBAC design, Pod Security and admission policy, secrets encryption, multi-tenancy, CIS hardening and incident response.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Threat model and the layers of cluster security (12 min)
2. Authentication, service accounts and token security (12 min)
3. RBAC design, least privilege and auditing (13 min)
4. Pod Security Standards and admission policy (13 min)
5. Secrets: encryption at rest and external stores (12 min)
6. Multi-tenancy and isolation (12 min)
7. API server and kubelet hardening with the CIS benchmark (13 min)
8. Incident response for a compromised cluster (14 min)
Cheat sheet

Lesson 1. Threat model and the layers of cluster security

12 minutes. Build a threat model for a cluster, map attacker paths from code to control plane, and see which controls break each path.

Security work is easier when it starts from how attacks actually proceed. A cluster has several layers, each with its own entry points, and the strongest control is usually the one that stops the attacker at the first step.

What you will learn

- The layers: cloud, cluster, container and code
- Common attack paths in Kubernetes
- Assets and trust boundaries
- Mapping controls to attack steps
- Where the CKS domains fit

The layers

The cloud native security model describes four layers, cloud or infrastructure, cluster, container and code. Each layer builds on the one below, so a weakness lower down undermines controls above it. A hardened container on a node whose kubelet accepts anonymous requests is still exposed.

Attack paths

A typical path begins with a vulnerable application or a leaked credential. The attacker gets code execution in a pod, reads the mounted service account token, calls the API server, and tries to escalate through permissive RBAC, a privileged pod, a host mount or the cloud metadata service. From a node they can reach other pods, secrets and cloud credentials. Another path begins with an exposed dashboard, kubelet or etcd port.

Assets and boundaries

List what is valuable: secrets, customer data, the ability to run code on nodes, cloud credentials, the CI pipeline and the images it produces. Mark trust boundaries: internet to ingress, pod to pod across namespaces, workload to API server, node to cloud, and build to runtime. Controls belong at those boundaries.

Mapping controls

For each step in a path, name the control that stops or detects it. Image scanning and minimal images reduce the chance of the first foothold. Dropping unnecessary capabilities and running as non-root limits what code can do. Disabling token automount denies the API foothold. Least privilege RBAC limits escalation. Network policies and metadata blocking limit movement. Audit logs and runtime detection find what slipped through.

```
path = [
    ("exploit app", "image scan, patching"),
    ("read service account token", "automount off, short lived tokens"),
    ("call API server", "least privilege RBAC"),
    ("escape to node", "pod security restricted, seccomp, AppArmor"),
    ("move sideways", "network policy, metadata block"),
    ("hide", "audit logs, runtime detection"),
]
for step, control in path:
    print(step, "->", control)
```

MARGA by EknathaLabs marga.eknathalabs.com

How this maps to CKS

The CKS exam domains line up with the layers: cluster setup and hardening, system hardening, minimising microservice vulnerabilities, supply chain security and monitoring and runtime security. This module goes deep on the reasoning, and the exam practice module drills the tasks.

Diagram: Threat model and the layers of cluster security

Part	What it does	What goes wrong
Application code	Entry point.	Known vulnerability.
Pod and container	Foothold.	Root, caps.
API server	Escalation target.	Broad RBAC.
Node and host	Escape target.	Privileged pod.
Cloud credentials	Wider impact.	Metadata reachable.

Flow: Application code to Pod and container / Pod and container to API server / Pod and container to Node and host / Node and host to Cloud credentials

Incident: From one vulnerable pod to the cloud account

Incident: *A compromised web pod ended with access to a cloud storage bucket.*

```
$ kubectl exec web-1 -- cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
shop
$ kubectl exec web-1 -- curl -s -m 2 http://169.254.169.254/latest/meta-data/iam/security-credenti \
    als/
node-role
(this is a simulated illustration, not a real incident)
```

The pod could read its service account token and also reach the cloud metadata endpoint, which returned the node role credentials. The attacker used those to read a storage bucket. Disabling token automount for pods that do not need it, blocking the metadata address with network policy and using workload identity instead of node roles would each have cut the path. Layers of controls matter because the first one often fails.

Common mistakes

- Securing the cluster but not the cloud role
- Leaving default service account tokens mounted
- Treating the metadata endpoint as harmless
- Only scanning images
- Not writing down the attack paths you defend

Check yourself

Why model attack paths?

Answer: To place controls where they break the path

Which is a typical first foothold?

Answer: A vulnerable application or leaked credential

What does blocking the metadata address prevent?

Answer: Pods obtaining node cloud credentials

Which layer sits below the cluster layer?

Answer: Cloud or infrastructure

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 2. Authentication, service accounts and token security

12 minutes. Understand how users and workloads authenticate, how projected service account tokens work, and how to reduce the damage of a stolen token.

Kubernetes has no built-in user database. Humans arrive with certificates or tokens from an identity provider, and workloads arrive as service accounts. Most real incidents involve a credential that was too powerful, too long lived or in the wrong place.

What you will learn

- How users are authenticated: certificates, OIDC and webhooks
- Service accounts and the tokens pods receive
- Bound, audience scoped, short lived tokens
- Turning off automount and avoiding default accounts
- Anonymous access and kubeconfig hygiene

Users

Users are not objects in the API. A client certificate signed by the cluster CA gives a user name from its common name and groups from its organisation. This is simple, but certificates cannot be revoked individually, so avoid them for people. OpenID Connect integrates with an identity provider, giving short lived tokens and central offboarding, and is the usual choice. Static token files and basic auth are legacy and should be off.

Service accounts

A ServiceAccount is the identity of a workload. Each namespace has a default one. Pods use it unless told otherwise. Current clusters issue projected tokens: they are bound to the pod, expire in about an hour by default, are refreshed by the kubelet and carry an audience. Older long lived secret based tokens exist only if created deliberately, and should be avoided.

Reducing exposure

Most pods never call the API server, so they should not carry a token. Set `automountServiceAccountToken: false` on the service account or the pod. Create dedicated service accounts per workload with only the roles they need, never rely on the default account, and never bind powerful roles to it. Where a token is needed for another system, request an audience scoped projected token.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: reporter
  namespace: shop
automountServiceAccountToken: false
```

Check what is exposed

```
kubectl get sa -A
kubectl auth can-i --list --as=system:serviceaccount:shop:default -n shop
kubectl get secrets -A --field-selector type=kubernetes.io/service-account-token
kubectl create token reporter -n shop --duration=10m
```

The last command creates a short lived token for testing.

Anonymous access and kubeconfigs

Set `--anonymous-auth=false` on the API server unless a health check needs it, and on the kubelet. Treat kubeconfig files as credentials: keep them out of repositories, give each person their own, prefer short lived exec plugins from the cloud or OIDC, and rotate admin credentials after staff changes.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Authentication, service accounts and token security

Part	What it does	What goes wrong
Human user	OIDC or certificate.	Shared admin config.
ServiceAccount	Workload identity.	Default account.
Projected token	Bound and short lived.	Long lived secret.
API server	Authenticates.	Anonymous on.

Flow: Human user to API server / ServiceAccount to Projected token / Projected token to API server

Incident: A token in a public repository

Incident: *A developer committed a kubeconfig with a long lived admin credential.*

```
$ kubectl auth whoami
ATTRIBUTE   VALUE
Username    kubernetes-admin
Groups      [system:masters system:authenticated]
$ kubectl get secrets -A --field-selector type=kubernetes.io/service-account-token | wc -l
14
(this is a simulated illustration, not a real incident)
```

The committed file held a client certificate in the system:masters group, which cannot be revoked without replacing the CA. Anyone who copied it had permanent cluster admin. The team rotated the cluster credentials, moved people to OIDC with short lived tokens, deleted legacy token secrets and added secret scanning to the repository. Prefer identities that can be revoked and that expire.

Common mistakes

- Using client certificates for people
- Leaving automountServiceAccountToken on by default
- Binding roles to the default service account
- Keeping legacy long lived token secrets
- Committing kubeconfig files

Check yourself

What is a projected service account token?

Answer: A short lived token bound to a pod and an audience

How do you stop a pod receiving an API token?

Answer: Set automountServiceAccountToken to false

Why avoid client certificates for people?

Answer: They cannot be revoked individually

What should the default service account have?

Answer: No extra permissions

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. RBAC design, least privilege and auditing

13 minutes. Design roles and bindings that grant least privilege, spot dangerous permissions, and audit who can do what.

RBAC is simple to write and easy to over-grant. A few permissions are effectively cluster admin even though they look harmless, and reviewing roles for them is one of the highest value security tasks.

What you will learn

- The RBAC objects and how they combine
- Patterns: narrow roles, namespaced bindings, aggregated roles
- Dangerous permissions and privilege escalation paths
- Auditing with can-i, who-can tools and the audit log
- Process for requesting and reviewing access

Objects

A Role grants permissions within a namespace, and a ClusterRole grants cluster wide or reusable permissions. A RoleBinding binds a role or cluster role to subjects within one namespace, and a ClusterRoleBinding does so for the whole cluster. Binding a ClusterRole with a RoleBinding is a good pattern: define the permission set once and grant it per namespace. Rules list API groups, resources, verbs and optionally resource names.

Design

Start from what the job needs and grant the verbs and resources for it, not a wildcard. Prefer groups from your identity provider as subjects over individual names. Use one namespace per team and bind built-in roles such as view and edit there. Keep admin and cluster-admin for a small set of break-glass accounts, with the use audited.

```
kubectl create role pod-reader --verb=get,list,watch --resource=pods -n shop
kubectl create rolebinding jane-reads --role=pod-reader --user=jane -n shop
kubectl auth can-i delete pods -n shop --as jane
```

Dangerous permissions

Some permissions allow escalation. The verbs escalate and bind on roles let someone grant themselves more. Creating pods in a namespace lets a user mount any secret there, run privileged containers if policy allows, or use any service account in the namespace. Reading secrets exposes credentials. Access to `pods/exec` gives a shell in workloads. Impersonation lets a subject act as another. Wildcards in verbs or resources, and any permission on `nodes/proxy`, deserve special review.

Auditing

List who can do something with `kubectl auth can-i --list --as`, or a plugin such as kubectl-who-can or rbac-tool to answer "who can create pods" across the cluster. Review ClusterRoleBindings to cluster-admin and system:masters regularly. The audit log shows which identities actually used permissions, which helps remove unused ones.

```
kubectl get clusterrolebindings -o json | grep -B2 -A8 cluster-admin | head -40
```

Process

Request access through a reviewed change in version control, grant it with an expiry where possible and remove it when the need ends. Review roles each quarter. Treat a new role that includes secrets, exec, escalate, bind or wildcards as needing a second reviewer.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: RBAC design, least privilege and auditing

Part	What it does	What goes wrong
Role or ClusterRole	Defines permissions.	Wildcards.
Binding	Grants to subjects.	Cluster wide by mistake.
Group or user	Receives access.	Individuals.
Audit	Finds excess.	Never reviewed.

Flow: Role or ClusterRole to Binding / Binding to Group or user / Group or user to Audit

Incident: The read-only role that could escalate

Incident: *A role called viewer turned out to let its holders become administrators.*

```
$ kubectl get clusterrole viewer -o jsonpath='{.rules}'
[{"apiGroups":["*"],"resources":["pods","secrets"],"verbs":["get","list","create"]}]
$ kubectl auth can-i create pods -n kube-system --as viewer-user
yes
(this is a simulated illustration, not a real incident)
```

The role allowed create on pods and get on secrets across the cluster, so a holder could start a pod in any namespace, mount any secret or use any service account. The name hid the risk. Replacing it with a narrow role bound only in specific namespaces, and reviewing roles for create on pods, secrets access, exec, escalate and bind, removed the path.

Common mistakes

- Wildcards in verbs or resources
- Granting create pods without considering what pods can mount
- Using ClusterRoleBindings where RoleBindings suffice
- Binding to individuals instead of groups
- No periodic review

Check yourself

What does a RoleBinding to a ClusterRole do?

Answer: Grants that permission set within one namespace

Why is create on pods dangerous?

Answer: A pod can mount secrets and use service accounts in its namespace

Which verbs allow privilege escalation in RBAC?

Answer: escalate and bind

How do you check what a user can do?

Answer: kubectl auth can-i --list --as the user

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Pod Security Standards and admission policy

13 minutes. Apply Pod Security admission with the privileged, baseline and restricted standards, and add policy engines for rules the standards do not cover.

A pod spec contains settings that decide how much of the node a container can touch. Pod Security Standards describe safe settings, and admission enforces them, so risky pods are stopped before they run.

What you will learn

- The three standards and what each forbids
- Pod Security admission modes: enforce, audit and warn
- Rolling out safely with labels per namespace
- Writing a compliant pod with a security context
- Policy engines: ValidatingAdmissionPolicy, Kyverno, Gatekeeper

The standards

Privileged is unrestricted and meant for system components. Baseline blocks known escalations: privileged containers, host namespaces, host paths and unsafe capabilities. Restricted adds current hardening practice: run as non-root, no privilege escalation, drop all capabilities except a short list, a RuntimeDefault or Localhost seccomp profile and limited volume types.

Admission modes

The Pod Security admission controller is built in and configured with namespace labels. Mode enforce rejects violating pods, audit records violations in the audit log and warn returns a warning to the user. Each mode has a level and a version label.

```
kubectl label ns shop pod-security.kubernetes.io/enforce=restricted pod-security.kubernetes.io/enf \
    orce-version=latest
kubectl label ns shop pod-security.kubernetes.io/warn=restricted pod-security.kubernetes.io/audit= \
    restricted
kubectl get ns shop --show-labels
```

Rolling out

Introduce it gradually. Label namespaces with warn and audit at the target level first and read the output, fix workloads, then switch on enforce. System namespaces such as kube-system usually need privileged. Because enforcement is per namespace, new namespaces need labels by default, which a platform can automate.

A compliant pod

A restricted pod sets non-root, disables escalation, drops capabilities and uses the runtime default seccomp profile.

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

MARGA by EknathaLabs marga.eknathalabs.com

Beyond the standards

The standards are fixed. Organisation rules such as allowed registries, required labels or banned host ports need a policy engine. ValidatingAdmissionPolicy, built in and written in CEL, covers many rules without running a webhook. Kyverno and OPA Gatekeeper add richer policy languages, mutation and reporting. Keep policy as code in version control and test it before rollout.

Diagram: Pod Security Standards and admission policy

Part	What it does	What goes wrong
Namespace labels	Select level and mode.	Missing labels.
Pod Security admission	Checks each pod.	Warn only.
Policy engine	Custom rules.	Webhook down.
Pod spec	Security context.	Root and escalation.

Flow: Namespace labels to Pod Security admission / Pod Security admission to Policy engine / Policy engine to Pod spec

Incident: A namespace nobody labelled

Incident: *A privileged pod ran in a new namespace because the standard was only applied to old ones.*

```
$ kubectl get ns dev-new --show-labels
NAME      STATUS   AGE   LABELS
dev-new   Active   2d    kubernetes.io/metadata.name=dev-new
$ kubectl get pod debug -n dev-new -o jsonpath='{.spec.containers[0].securityContext}'
{"privileged":true}
(this is a simulated illustration, not a real incident)
```

Pod Security applies per namespace through labels, and the new namespace had none, so nothing was enforced. The team added a namespace creation workflow that sets warn, audit and enforce labels, and a policy that rejects namespaces without them. A control that must be remembered per namespace will eventually be forgotten.

Common mistakes

- Enforcing restricted without a warn phase
- Leaving new namespaces unlabelled
- Using the standards for organisation specific rules
- Running a policy webhook without exclusions
- Forgetting readOnlyRootFilesystem and capability drops

Check yourself

Which standard forbids running as root?

Answer: Restricted

How is Pod Security admission configured?

Answer: By labels on namespaces

What does warn mode do?

Answer: Returns a warning to the user without rejecting

What can express custom rules without a webhook?

Answer: ValidatingAdmissionPolicy

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Secrets: encryption at rest and external stores

12 minutes. Protect Secret objects with encryption at rest and tight access, and decide when to move to an external secret manager.

Kubernetes Secrets are base64 encoded, not encrypted. Out of the box anyone who can read etcd, a backup or the Secret object itself can read the value. Several layers fix that, and each addresses a different threat.

What you will learn

- What a Secret protects and what it does not
- Encryption at rest with an EncryptionConfiguration and KMS
- Access control and avoiding exposure through pods
- External secret stores and the CSI driver or operators
- Rotation after exposure

What base64 gives you

A Secret stores values as base64 so that binary data fits in YAML. Decoding takes one command. The protection comes from RBAC on the API, from encryption of the stored bytes and from keeping Secrets out of places where they leak, such as logs, environment dumps, images and version control.

Encryption at rest

The API server can encrypt chosen resources before writing them to etcd, using an EncryptionConfiguration file passed with `--encryption-provider-config`. Providers include aescbc and secretbox with a local key, and kms, which wraps a data key with a key in an external key management service, so the key never sits next to the data. The first provider in the list is used to write, and the others are for reading. After enabling it, rewrite existing Secrets so they are encrypted.

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources: [secrets]
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: BASE64_32_BYTE_KEY
      - identity: {}
```

```
kubectl get secrets -A -o json | kubectl replace -f -
```

Verify

Read a Secret directly from etcd and check for the encryption prefix instead of readable text. An encrypted value starts with `k8s:enc:` and the provider name.

```
ETCDCTL_API=3 etcdctl get /registry/secrets/shop/db --endpoints=https://127.0.0.1:2379 --cacert=/e \
    tc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes \
    /pki/etcd/server.key | head -c 80
```

Reducing exposure

Restrict get, list and watch on Secrets, since list returns every value. Mount Secrets as files instead of environment variables when possible, because environment variables appear in process listings and crash dumps. Limit who can create pods in a namespace, because a pod can mount any Secret there. Never put Secrets in images or manifests in version control unencrypted.

MARGA by EknathaLabs marga.eknathalabs.com

External stores

Tools such as the External Secrets Operator and the Secrets Store CSI driver fetch values from a manager such as Vault or a cloud secrets service and present them to pods, with central rotation, audit and access policy. They reduce the amount of long lived material in etcd. The trade is dependency on the external service and its availability. After any exposure, rotate the secret itself and everything that used it, since removing access does not undo a copy.

Diagram: Secrets: encryption at rest and external stores

Part	What it does	What goes wrong
Pod	Mounts the secret.	Env var leak.
API server	Encrypts on write.	No config.
etcd	Stores ciphertext.	Plain text.
KMS or external store	Holds the key.	Unavailable.

Flow: Pod to API server / API server to etcd / API server to KMS or external store

Incident: Plain text in a backup

Incident: *A routine review found a database password readable in an etcd snapshot.*

```
$ strings /backup/etcd-snapshot.db | grep -m1 -i password
password: Sup3rS3cret
$ grep -c encryption-provider-config /etc/kubernetes/manifests/kube-apiserver.yaml
0
(this is a simulated illustration, not a real incident)
```

Without an encryption configuration, Secrets were stored in readable form, and snapshots, stored on a shared disk, exposed them. The team enabled encryption at rest, rewrote all Secrets, rotated the exposed credentials and encrypted and restricted backups. Enabling encryption protects new writes only until existing objects are rewritten.

Common mistakes

- Believing base64 is encryption
- Enabling encryption and not rewriting existing Secrets
- Granting list on Secrets broadly
- Passing secrets as environment variables everywhere
- Not rotating after exposure

Check yourself

Is a Kubernetes Secret encrypted by default?

Answer: No, it is base64 encoded

What must you do after enabling encryption at rest?

Answer: Rewrite existing Secrets

What does a KMS provider improve?

Answer: The key is kept outside the cluster storage

Why can list on Secrets be dangerous?

Answer: It returns the values of all Secrets in scope

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Multi-tenancy and isolation

12 minutes. Compare namespace, node and cluster level isolation for tenants, and combine RBAC, quotas, policy, network and runtime isolation to match the trust level.

Sharing a cluster saves money and effort, but it shares the kernel, the control plane and the network. How much isolation you add depends on how much you trust the tenants, and the options form a ladder.

What you will learn

- Soft and hard multi-tenancy
- Namespaces as the unit and what they do not isolate
- Node pools, sandboxed runtimes and virtual clusters
- Isolating network, secrets, resources and policy
- Choosing the level

Soft and hard

Soft multi-tenancy assumes tenants are cooperating teams in one organisation who might make mistakes. Hard multi-tenancy assumes a tenant may be hostile, as in a service offered to outside customers. Kubernetes can give strong soft isolation easily. Hard isolation inside one cluster is difficult because containers share a kernel.

Namespaces

A namespace scopes names, RBAC, quotas and policies. It does not by itself isolate the network, nodes or kernel. Pair each tenant namespace with a RoleBinding to the tenant group, a ResourceQuota and LimitRange, default deny network policies, Pod Security labels and no access to other tenants Secrets. Disable service account token automount and cross namespace service access where not needed.

Stronger isolation

Dedicated node pools with taints and node selectors keep tenant pods off shared nodes and limit the effect of a container escape. Sandboxed runtimes such as gVisor or Kata Containers, selected with a RuntimeClass, add a stronger boundary between the container and the host kernel. Virtual clusters give each tenant its own API server and control plane objects on shared nodes, reducing control plane sharing. Separate clusters give the strongest isolation at the highest cost.

```
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: sandboxed
handler: runsc
```

Shared components

Watch the things that are shared: cluster scoped resources such as CRDs and ClusterRoles, admission webhooks, the DNS and ingress controllers, logging and monitoring, and the API server itself, where a noisy tenant can use up request capacity. Use API Priority and Fairness settings, quotas on object counts and per-tenant observability.

Choosing

For teams in one company, namespaces with the controls above are usually enough. For regulated or semi-trusted workloads, add node pools and a sandbox runtime. For untrusted code from outside parties, prefer separate clusters or hardware isolation. Document the model so that tenants know what they share.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: Multi-tenancy and isolation

Part	What it does	What goes wrong
Namespace controls	RBAC, quota, policy.	Missing one.
Network isolation	Default deny.	Open by default.
Node pool	Dedicated nodes.	Shared nodes.
Sandbox runtime	Kernel boundary.	Not enabled.
Separate cluster	Strongest isolation.	Cost.

Flow: Namespace controls to Network isolation / Namespace controls to Node pool / Node pool to Sandbox runtime / Sandbox runtime to Separate cluster

Incident: A neighbour that starved the API server

Incident: *One tenant controller listing every pod in a tight loop made other tenants deployments slow.*

```
$ kubectl get --raw /metrics | grep apiserver_flowcontrol_current_inqueue_requests | head -3
apiserver_flowcontrol_current_inqueue_requests{priority_level="workload-low"} 164
$ kubectl get flowschemas | grep tenant
(no output)
(this is a simulated illustration, not a real incident)
```

The tenant tool shared a default flow with everyone else, and its heavy list calls queued other requests behind it. Adding a flow schema that placed the tenant in its own priority level, and fixing the tool to use a watch, restored fair service. Shared control plane capacity is a multi-tenancy concern, not only pods and networks.

Common mistakes

- Treating namespaces as a security boundary on their own
- Missing default deny network policy per tenant
- Sharing nodes for untrusted workloads
- Ignoring noisy neighbours on the API server
- Not documenting what is shared

Check yourself

Which assumption defines hard multi-tenancy?

Answer: Tenants may be hostile

What does a namespace not isolate by itself?

Answer: The network, nodes and kernel

What adds a stronger boundary between containers and the host kernel?

Answer: A sandboxed runtime such as gVisor selected by RuntimeClass

Which option gives the strongest isolation?

Answer: Separate clusters

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. API server and kubelet hardening with the CIS benchmark

13 minutes. Run kube-bench, understand the most important API server, kubelet and etcd settings, and fix findings without breaking the control plane.

The CIS Kubernetes Benchmark is a long list of configuration checks. Tools run it for you, but you still have to understand a finding well enough to fix it safely, because the control plane settings live in static pod manifests where a typo stops the API server.

What you will learn

- How to run kube-bench and read its output
- Important API server flags
- Kubelet settings that matter
- etcd and file permission checks
- A safe edit procedure

Running the benchmark

kube-bench is an open source tool that runs the checks as a job or on the node and prints PASS, FAIL, WARN and INFO for each numbered recommendation, with a remediation text. Run it on control plane and worker nodes, because they have different checks.

```
kube-bench run --targets master,node,etcd,policies
kube-bench run --targets master --check 1.2.1
```

API server settings

Key settings include anonymous authentication off, authorization mode Node and RBAC, never AlwaysAllow, the NodeRestriction admission plugin enabled, audit logging on with a policy, profiling off, TLS cipher and version settings, the service account issuer and key files set, and etcd client certificates used. Insecure ports must be closed. The manifest at `/etc/kubernetes/manifests/kube-apiserver.yaml` holds these as command flags.

Kubelet settings

On each node the kubelet configuration should set `authentication.anonymous.enabled: false`, `authorization.mode: Webhook`, a client CA file for x509 authentication, `readOnlyPort: 0` to disable the unauthenticated read-only port, `protectKernelDefaults: true`, `rotateCertificates: true` and event rate limits. A kubelet that accepts anonymous requests with AlwaysAllow lets anyone who can reach port 10250 run commands in pods.

```
grep -E "anonymous|mode|readOnlyPort|protectKernelDefaults" /var/lib/kubelet/config.yaml
sudo systemctl restart kubelet
```

etcd and files

etcd must use client certificate authentication, peer TLS, and a data directory owned by etcd with mode 700. Configuration files and certificates under `/etc/kubernetes` should be owned by root with restrictive permissions, and the kubeconfig files not world readable.

A safe procedure

Copy the manifest before editing, change one flag at a time, then watch the static pod restart with `crictl ps` and check that `kubectl get nodes` still works. If the API server does not return, read its container logs and restore the copy. Re-run the single failed check afterward to confirm the fix, and keep a record of exceptions you accept with their reason.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: API server and kubelet hardening with the CIS benchmark

Part	What it does	What goes wrong
kube-bench	Reports findings.	Wrong target.
API server flags	Auth, audit, admission.	Typo in manifest.
Kubelet config	Anonymous off.	Not restarted.
etcd and files	TLS and permissions.	World readable.
Re-run check	Prove the fix.	Skipped.

Flow: kube-bench to API server flags / kube-bench to Kubelet config / kube-bench to etcd and files / API server flags to Re-run check / Kubelet config to Re-run check

Incident: The audit flag that stopped the API server

Incident: *An engineer enabled audit logging and the API server did not return.*

```
$ sudo crictl ps -a | grep kube-apiserver
f3a1c9  Exited  kube-apiserver  4 minutes ago
$ sudo crictl logs f3a1c9 2>&1 | tail -2
Error: loading audit policy file: open /etc/kubernetes/audit-policy.yaml: no such file or director \
    y
(this is a simulated illustration, not a real incident)
```

The flags were added, but the policy file was not mounted into the static pod, so the process exited at start and the kubelet kept restarting it. Restoring the backed up manifest recovered the API, and the engineer re-applied the change with the volume and mount for the policy and log paths. Always copy the manifest first, and remember that files used by the API server must be mounted into the pod.

Common mistakes

- Editing a manifest without a backup
- Adding file flags without volume mounts
- Fixing many findings at once
- Running only the control plane checks
- Accepting findings without recording the reason

Check yourself

What does kube-bench do?

Answer: Checks cluster configuration against the CIS benchmark

Which kubelet setting turns off unauthenticated access?

Answer: authentication.anonymous.enabled set to false

Where do API server flags live on a kubeadm cluster?

Answer: In the kube-apiserver static pod manifest

What must accompany a flag that points to a file?

Answer: A volume and mount that makes the file visible in the pod

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Incident response for a compromised cluster

14 minutes. Detect, contain, investigate and recover from a compromised pod or node using audit logs, runtime detection and a clear order of actions.

When a workload is compromised, the pressure is to delete it. Deleting destroys evidence and may leave the real problem untouched. A short, practised order of actions contains the damage first and investigates second.

What you will learn

- Signals: audit logs, Falco alerts, network and cost anomalies
- Containment without destroying evidence
- Investigation steps and what to collect
- Eradication and credential rotation
- Lessons and detection improvements

Detection

Signals come from several places. Falco or another runtime sensor alerts on a shell spawned in a container, writes below /etc, unexpected outbound connections and reads of sensitive files. The audit log shows unusual API calls, such as listing secrets or creating privileged pods from a service account that normally does not. Cost or network graphs may show cryptomining or data exfiltration. Treat an alert as a hypothesis and confirm it.

Contain first

Limit the blast radius before you investigate. Apply a network policy that isolates the pod, label it so a quarantine policy matches, and cordon the node so nothing new lands there. Revoke or rotate the service account token and any credential the pod could read. Preserve evidence: do not delete the pod yet. Take a snapshot of the node disk if you can, and copy logs.

```
kubectl label pod web-7d9 quarantine=true --overwrite
kubectl cordon node-a3
kubectl get pod web-7d9 -o yaml > /secure/evidence/web-7d9.yaml
kubectl logs web-7d9 --all-containers > /secure/evidence/web-7d9.log
```

Investigate

Find out how the attacker got in, what they touched and whether they moved. Review audit logs for that service account and for the node, list what the pod could access with its permissions, check for new pods, jobs, cron jobs and RBAC objects, inspect the image and the running process tree, and check cloud audit logs for the node role. Build a timeline.

Eradicate and recover

Remove the cause: patch the vulnerability, rebuild the image, redeploy from a known good source and replace affected nodes instead of cleaning them. Rotate every credential that was reachable: tokens, Secrets, database passwords and cloud keys. Verify with scans and detection that the attacker no longer has access, and watch closely for a while.

Learn

Write a short blameless review. Add the detection that would have caught the event earlier, close the path that was used, and rehearse the response. Keep a runbook with contacts, decisions about who may isolate production workloads and where evidence is stored.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Incident response for a compromised cluster

Part	What it does	What goes wrong
Detect	Falco, audit, graphs.	Not alerting.
Contain	Isolate and rotate.	Delete too early.
Investigate	Timeline and scope.	Partial view.
Recover	Rebuild and rotate.	Reused credentials.
Learn	New detection.	No review.

Flow: Detect to Contain / Contain to Investigate / Investigate to Recover / Recover to Learn

Incident: Deleting the pod and losing the trail

Incident: *A suspicious pod was deleted immediately, and the team could not tell what it had accessed.*

```
$ kubectl delete pod web-7d9
pod "web-7d9" deleted
$ kubectl get pods -l app=web
NAME      READY   STATUS    AGE
web-8c1   1/1     Running   4s
$ kubectl logs web-7d9
Error from server (NotFound): pods "web-7d9" not found
(this is a simulated illustration, not a real incident)
```

The replacement pod appeared at once, but the original process tree, files and logs were gone, and its service account token was still valid. The team could not tell what the attacker had read. In a later incident they isolated the pod with a network policy, rotated the token, copied evidence and then replaced it. Contain first, preserve evidence, then remove.

Common mistakes

- Deleting the workload before collecting evidence
- Not rotating the credentials the pod could reach
- Cleaning a node instead of replacing it
- No runtime detection or audit logs enabled
- No written response runbook

Check yourself

What is the first priority after confirming a compromise?

Answer: Contain it without destroying evidence

How can you isolate a pod quickly?

Answer: Label it to match a quarantine network policy

Why replace a node instead of cleaning it?

Answer: You cannot be sure the host is clean

Which credentials should be rotated?

Answer: Everything the compromised workload could reach

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Kubernetes security

RBAC audit commands, Pod Security labels, hardening flags and incident response steps.

Quick RBAC audit

Job	Command	
What can a user do	`kubectl auth can-i --list --as=USER -n NS`	
Who can read secrets	`kubectl auth can-i get secrets --as=USER -n NS`	
Find broad bindings	`kubectl get clusterrolebindings -o wide`	
Wildcard rules	`kubectl get clusterroles -o yaml	grep -B3 '*'`	

Pod Security labels

Label	Meaning
`pod-security.kubernetes.io/enforce`	Reject pods that break the standard
`pod-security.kubernetes.io/audit`	Record violations
`pod-security.kubernetes.io/warn`	Show warnings to the user

Hardening checks

Control	Check
Anonymous API access	`--anonymous-auth=false`
Authorization modes	`--authorization-mode=Node,RBAC`
Kubelet anonymous	`anonymous: enabled: false` in the kubelet config
Audit logging	`--audit-policy-file` and `--audit-log-path`
Benchmark	`kube-bench run --targets master,node`

Incident basics

Step	Action
Contain	Quarantine label, deny policy, cordon the node
Preserve	Copy logs and snapshot the node disk
Rotate	Tokens, secrets and cloud keys the pod could reach
Rebuild	Known good image and clean node

MARGA by EknathaLabs marga.eknathalabs.com